PPG Pendidikan Profesi Guru Calon Guru



PPG CALON GURU

disusun oleh :
Triana Elisabet Sinaga | 25105260168

2026

Bahan Bacaan Informatika

KELAS X SMA | FASE E

DAFTAR ISI MODUL

TOPIK I
BERPIKIR KOMPUTASIONAL 1

1.1 Konsep Berpikir Komputasional 1

1.2 Pondasi Berpikir Komputasional 2

TOPIK 2
DATA DAN INFORMASI 4

2.1 Pengertian Data & Informasi 4

2.2 Jenis-Jenis Data 5

2.3 Validitas Sumber Data 7

TOPIK 3
STRUKTUR DATA 12

3.1 Pengertian Struktur Data 13

3.2 Jenis-Jenis Struktur Data 14

TOPIK 4
ALGORITMA STANDAR 17

4.1 Konsep Dasar Algoritma 18

4.2 Algoritma Pencarian (Searching) 27

4.2 Algoritma Pengurutan (Sorting) 30

TOPIK 5
SISTEM KOMPUTER 35

5.1 Pengenalan Sistem Komputer 36

5.2 Interaksi Manusia dan Komputer 38

5.2 Sistem Operasi 39

TOPIK 6
PERAN SISTEM OPERASI 53

6.1 Jenis-Jenis Sistem Operasi 54

6.2 Instalasi Sistem Operasi 56

6.2 Peran Sistem Operasi 58

TOPIK 1
BERPIKIR KOMPUTASIONAL



Apa itu Berpikir Komputasional?

Berpikir komputasional adalah cara kita berpikir untuk memecahkan masalah dengan menggunakan konsep dan teknik yang digunakan dalam ilmu komputer. Sederhananya, ini adalah cara kita berpikir seperti seorang programmer saat menghadapi suatu masalah.

1. Mengapa Berpikir Komputasional Penting?

- Pemecahan Masalah: Membantu kita berpikir logis dan sistematis untuk menemukan solusi terbaik.
- Kreativitas: Melatih kita untuk berpikir "out of the box" dan menemukan cara-cara baru untuk menyelesaikan masalah.
- Keterampilan Abad 21: Sangat relevan dengan tuntutan dunia kerja saat ini yang semakin berbasis teknologi.

2. Pondasi Berpikir Komputasional

1. Dekomposisi

Dekomposisi adalah memecah masalah membagi masalah besar menjadi bagian-bagian kecil agar lebih mudah dikelola.

Contoh:

Membuat secangkir teh bisa dipecah menjadi beberapa langkah kecil:

- Menyiapkan teko
- Merebus air
- Menyiapkan kantong teh
- Mencampurkan air panas dan kantong teh
- Menunggu beberapa saat
- Menuangkan teh ke cangkir
- Menambahkan gula atau susu (opsional) Dengan memecah masalah menjadi langkah-langkah kecil, tugas menjadi lebih mudah dikelola.

2. Pengenalan Pola

Pengenalan pola adalah menemukan kesamaan atau pola dalam sesuatu.

Contoh:

Setiap kali membuat teh, kamu akan melakukan langkah-langkah yang sama. Ini adalah pola yang bisa kamu kenali.

- Mengidentifikasi variabel: Kamu bisa menyesuaikan jumlah air, jenis teh, atau tambahan gula sesuai selera. Ini adalah variabel yang bisa diubah-ubah dalam pola tersebut.

3. Abstraksi

Abstraksi adalah fokus pada hal penting atau mengabaikan hal yang tidak penting

Contoh:

- Hal penting dalam membuat teh adalah adanya air panas, teh, dan wadah untuk mencampurnya. Kamu tidak perlu memikirkan merek teko atau jenis kompor yang digunakan.
- Mengabaikan detail: Detail seperti suhu air yang tepat atau waktu perendaman teh bisa diabaikan jika kamu hanya ingin membuat teh yang sederhana.

4. Algoritma:

Algoritma adalah membuat langkah-langkah yang jelas dan berurutan untuk mencapai tujuan.

Contoh:

Kamu bisa membuat sebuah algoritma sederhana untuk membuat teh:

- Isi teko dengan air.
- Nyalakan kompor.
- Masukkan kantong teh ke dalam teko setelah air mendidih.
- Tunggu 3-5 menit.
- Tuangkan teh ke dalam cangkir. Dengan mengikuti algoritma ini, kamu akan selalu bisa membuat secangkir teh yang enak.

TOPIK 2
DATA DAN INFORMASI



Dalam kehidupan sehari-hari, kita sering dihadapkan dengan berbagai macam data dan informasi. Data merupakan bahan mentah yang perlu diolah menjadi informasi agar dapat bermanfaat. Informasi yang akurat dan kredibel sangat penting untuk pengambilan keputusan yang tepat.

A. Pengertian Data & Informasi

- Data adalah kumpulan fakta, angka, simbol, atau observasi yang belum diolah dan belum memiliki makna yang jelas bagi penggunanya. Data bisa berupa angka, teks, gambar, suara, atau video.

- Informasi adalah data yang telah melalui proses pengolahan, sehingga memiliki makna dan berguna untuk pengambilan keputusan. Informasi dapat berupa laporan, grafik, tabel, atau kesimpulan dari suatu penelitian.

B. Jenis-Jenis Data

Data dapat dikategorikan berdasarkan berbagai aspek. Berikut adalah beberapa klasifikasi data yang umum digunakan:

Berdasarkan Sifatnya:

- Data Kualitatif: Data yang bersifat deskriptif, tidak bisa diukur dengan angka secara langsung. Data ini lebih kepada kualitas atau ciri khas dari suatu objek.
- Contoh: Warna, rasa, pendapat, jenis kelamin.
- Data Kuantitatif: Data yang bersifat numerik, dapat diukur dan dinyatakan dalam angka.
- Contoh: Usia, tinggi badan, berat badan, suhu.

Berdasarkan Skala Pengukuran:

- Data Nominal: Data yang hanya berfungsi sebagai label atau kategori. Tidak ada urutan atau hierarki di antara kategori-kategori tersebut.
    - Contoh: Jenis kelamin (laki-laki, perempuan), golongan darah (A, B, AB, O).
- Data Ordinal: Data yang menunjukkan peringkat atau urutan. Ada hierarki di antara kategori, namun jarak antar kategori tidak selalu sama.
    - Contoh: Tingkat pendidikan (SD, SMP, SMA, S1), peringkat dalam lomba.

- Data Interval: Data yang memiliki jarak yang sama antara setiap titik datanya. Namun, tidak memiliki titik nol absolut.
  - Contoh: Suhu dalam Celcius atau Fahrenheit, tahun.
- Data Rasio: Data yang memiliki titik nol absolut dan jarak yang sama antara setiap titik datanya. Perbandingan antara dua nilai data rasio memiliki makna.
  - Contoh: Usia, berat badan, pendapatan.

Berdasarkan Sumbernya:

- Data Primer: Data yang dikumpulkan langsung oleh peneliti atau pengumpul data.
  - Contoh: Hasil wawancara, hasil observasi, hasil eksperimen.
- Data Sekunder: Data yang telah dikumpulkan oleh pihak lain dan kemudian digunakan kembali.
  - Contoh: Data sensus penduduk, laporan keuangan perusahaan, data dari database publik.

Berdasarkan Waktu Pengumpulan:

- Data Time Series: Data yang dikumpulkan dalam rentang waktu tertentu, sehingga menunjukkan perubahan data dari waktu ke waktu.
  - Contoh: Harga saham harian, tingkat inflasi tahunan.
- Data Cross-Sectional: Data yang dikumpulkan pada suatu titik waktu tertentu dari berbagai sumber yang berbeda.
  - Contoh: Data penjualan produk pada bulan Januari dari berbagai cabang perusahaan.

C. Validitas Sumber Data

Validitas sumber data mengacu pada sejauh mana data yang diperoleh berasal dari sumber yang dapat dipercaya dan akurat. Data yang tidak valid dapat menghasilkan informasi yang menyesatkan dan berujung pada keputusan yang keliru.

Mengapa Validitas Sumber Data Penting?

- Pengambilan Keputusan: Data yang valid akan menghasilkan keputusan yang lebih baik dan tepat sasaran.
- Kualitas Penelitian: Data yang valid akan meningkatkan kualitas penelitian dan hasil yang diperoleh.
- Kepercayaan Publik: Data yang valid akan meningkatkan kepercayaan publik terhadap informasi yang disajikan

Dalam kesimpulan, validitas sumber data adalah aspek penting dalam penelitian dan pengambilan keputusan. Dengan memahami konsep validitas dan cara menjaminnya, kita dapat meningkatkan kualitas informasi yang kita gunakan.

Contoh Sumber Data yang Kurang Valid

1. **Data Anecdotal:**
    - Contoh: Cerita atau pengalaman pribadi yang tidak didukung oleh data kuantitatif.
    - Alasan: Data ini seringkali bias dan sulit digeneralisasikan karena hanya mewakili kasus-kasus individu.
2. **Data yang Tidak Terverifikasi:**
    - Contoh: Informasi yang beredar di media sosial tanpa sumber yang jelas atau data yang tidak melalui proses verifikasi.

3. **Data yang Ketinggalan Zaman:**
    - Contoh: Data sensus penduduk yang sudah berusia 10 tahun.
    - Alasan: Kondisi sosial, ekonomi, dan demografi dapat berubah secara signifikan dalam jangka waktu tersebut, sehingga data menjadi tidak relevan.
4. **Data yang Bias:**
    - Contoh: Survei yang hanya dilakukan pada kelompok tertentu (misal, hanya melibatkan pelanggan yang puas) atau pertanyaan yang mengarahkan responden ke jawaban tertentu.
    - Alasan: Data ini tidak mewakili populasi secara keseluruhan dan dapat menghasilkan kesimpulan yang bias.
5. **Data yang Tidak Konsisten:**
    - Contoh: Data yang bertentangan dengan data dari sumber lain yang kredibel.
    - Alasan: Ketidakkonsistenan ini menunjukkan adanya kesalahan dalam pengumpulan atau analisis data.
6. **Data yang Tidak Relevan:**
    - Contoh: Menggunakan data pendapatan per kapita untuk mengukur tingkat kebahagiaan.
    - Alasan: Variabel yang digunakan tidak relevan dengan konsep yang ingin diukur.

Cara Menilai Validitas Sumber Data

Berikut adalah beberapa cara yang dapat Anda lakukan untuk menilai validitas sumber data:

1. **Periksa Kredibilitas Sumber:**
   - Siapa yang menerbitkan informasi ini? Apakah lembaga atau individu yang memiliki kredibilitas di bidang tersebut?
   - Apakah sumber tersebut memiliki reputasi yang baik? Periksa riwayat publikasi atau penelitian mereka.
2. **Evaluasi Konten:**
   - Apakah informasi tersebut logis dan masuk akal? Informasi yang terlalu bagus untuk menjadi kenyataan atau bertentangan dengan fakta yang sudah diketahui perlu diwaspadai.
   - Apakah informasi tersebut didukung oleh bukti yang kuat? Cari tahu apakah ada data, studi, atau sumber lain yang mendukung klaim tersebut.
   - Apakah informasi tersebut disajikan secara objektif? Hindari sumber yang terlalu bias atau berusaha membujuk pembaca untuk mengambil sikap tertentu.
3. **Cek Sumber Asli:**
   - Cari tahu sumber asli informasi tersebut. Jangan hanya mengandalkan informasi yang sudah dibagikan berulang kali.
   - Lakukan pencarian balik (reverse image search) untuk gambar atau video. Ini dapat membantu Anda menemukan sumber asli dan konteks yang lebih luas.

4. **Bandingkan dengan Sumber Lain:**
    - Cari informasi yang sama dari sumber lain yang terpercaya. Jika informasi yang sama ditemukan di beberapa sumber yang berbeda, maka kemungkinan besar informasi tersebut valid.
    - Perhatikan perbedaan pendapat. Jika ada perbedaan pendapat yang signifikan, cobalah untuk mencari penjelasan yang lebih mendalam.
5. **Periksa Tanggal Publikasi:**
    - Informasi yang sudah lama mungkin tidak lagi relevan. Teknologi dan pengetahuan terus berkembang, sehingga informasi yang dulu dianggap benar bisa saja sudah ketinggalan zaman.
6. **Waspadai Bahasa yang Provokatif:**
    - Bahasa yang provokatif, emosional, atau bombastis sering digunakan untuk memanipulasi pembaca. Informasi yang valid biasanya disajikan dengan bahasa yang jelas dan objektif.
7. **Periksa URL:**
    - Perhatikan domain website. Apakah domain tersebut terlihat profesional dan relevan dengan topik yang dibahas?
    - Hindari website dengan domain yang mencurigakan atau mengandung kata-kata yang tidak jelas.

Contoh Sumber Data yang Terpercaya:

- Lembaga pemerintah: Badan Pusat Statistik (BPS), Kementerian Kesehatan, dll.
- Media mainstream: Koran, televisi, atau situs berita yang memiliki reputasi baik.

- Jurnal ilmiah: Publikasi yang telah melalui proses peer review.
- Organisasi internasional: PBB, WHO, dll.

Tips Tambahan:

- Gunakan mesin pencari khusus untuk memeriksa fakta: Ada banyak mesin pencari yang dirancang khusus untuk memverifikasi informasi, seperti Google Search, Snopes, dan FactCheck.org.
- Bergabunglah dengan komunitas yang peduli dengan kebenaran: Diskusi dengan orang-orang yang kritis dan rasional dapat membantu Anda mengembangkan kemampuan untuk menilai informasi.
- Jangan mudah percaya pada informasi yang baru pertama kali Anda lihat. Selalu lakukan verifikasi sebelum membagikan informasi kepada orang lain.

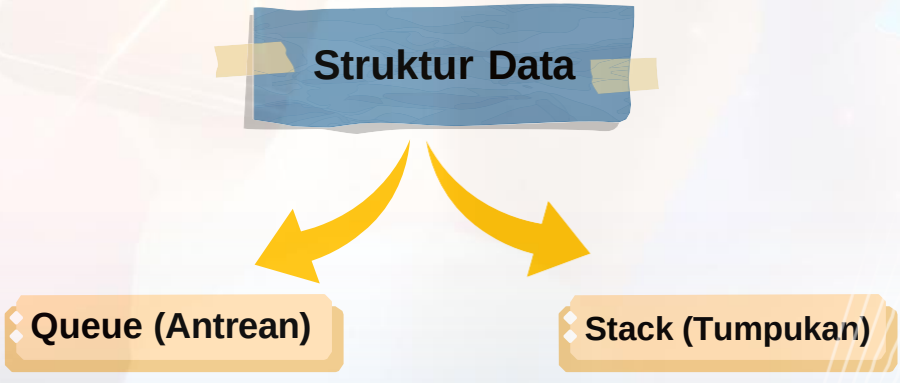
Dengan menerapkan langkah-langkah di atas, Anda dapat meningkatkan kemampuan Anda dalam menilai validitas sumber data dan menghindari penyebaran hoaks.

TOPIK 3
STRUKTUR DATA



Tujuan Pembelajaran:

1. Memahami konsep dasar struktur data dalam kehidupan sehari hari
2. Mampu menerapkan struktur data yang tepat untuk menyelesaikan masalah tertentu.



A. Pengertian Struktur Data



Struktur data bisa diibaratkan sebagai lemari pakaian. Lemari pakaian memiliki berbagai macam laci, gantungan, dan rak untuk menyimpan pakaian kita secara teratur. Nah, dalam pemrograman, struktur data juga berfungsi untuk menyimpan data dengan cara yang teratur dan mudah diakses.

Bayangkan sebuah program komputer:

- Data: Data bisa berupa angka, huruf, kata, atau bahkan data yang lebih kompleks seperti gambar atau video.
- Struktur data: Struktur data adalah wadah atau tempat untuk menyimpan data-data tersebut. Setiap struktur data memiliki cara tersendiri untuk mengatur dan menyimpan data.

Jadi intinya, struktur data adalah cara kita mengatur dan menyimpan data dalam program komputer. Dengan memilih struktur data yang tepat, kita dapat membuat program yang lebih efisien, terorganisir, dan mudah dipelihara.

Contoh sederhana:

Misalnya, kita ingin membuat program untuk mengelola daftar siswa. Kita bisa menggunakan struktur data array untuk menyimpan nama-nama siswa. Dengan menggunakan array, kita dapat dengan mudah mengakses data siswa berdasarkan nomor urutnya.

B. Jenis-Jenis Struktur Data

1. Queue)Antrean)



Struktur data queue merupakan teknik menyimpan data dimana data yang dimasukkan terlebih dahulu akan keluar duluan. Struktur data queue atau antrian bisa dibayangkan seperti antrian di kasir supermarket. Barang yang pertama kali dimasukkan ke dalam keranjang (masuk ke antrian) akan menjadi yang pertama dibayar (keluar dari antrian). Prinsip ini disebut FIFO (*First In, First Out*).

Contoh Sederhana:

Bayangkan kamu sedang mengantre untuk membeli tiket konser. Orang yang pertama kali datang akan menjadi yang pertama membeli tiket. Jika ada orang lain datang, mereka akan mengantre di belakang.

Dalam pemrograman, queue digunakan untuk:

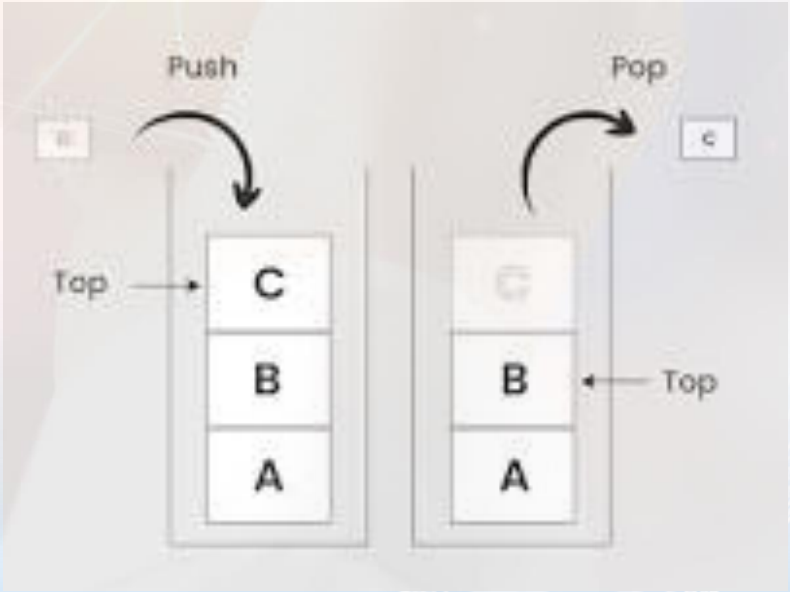
- Mengatur urutan tugas: Misalnya, dalam komputer, tugas-tugas yang masuk akan diantrekan untuk diproses.
- Membuat simulasi antrian: Misalnya, untuk membuat simulasi antrian di bank, restoran, atau sistem produksi.

2. Stack)Tumpukan)



Struktur data stack adalah sebuah teknik penyimpanan data yang bekerja berdasarkan prinsip LIFO (Last In, First Out). Artinya, data yang terakhir dimasukkan ke dalam stack akan menjadi data pertama yang diambil atau dikeluarkan. Bayangkan sebuah tumpukan piring: piring terakhir yang ditumpuk akan menjadi piring pertama yang diambil saat kita ingin menggunakannya.

Contoh ilustrasi:



Contoh Penggunaan Stack pada Komputer

- Browser: Ketika Anda mengklik tombol "Back" pada browser, browser akan mengambil alamat web terakhir yang Anda kunjungi dari stack. Alamat ini adalah alamat yang paling akhir dimasukkan ke dalam stack saat Anda membuka halaman tersebut.
- Undo/Redo: Fitur undo dan redo pada aplikasi pengolah kata atau gambar menggunakan stack. Setiap perubahan yang Anda lakukan akan disimpan dalam stack. Ketika Anda mengklik undo, perubahan terakhir akan dihapus dari stack.
- Panggilan Fungsi: Ketika sebuah fungsi memanggil fungsi lain, alamat kembalinya disimpan dalam stack. Hal ini memungkinkan program untuk kembali ke titik eksekusi yang tepat setelah fungsi yang dipanggil selesai dijalankan.



Tujuan Pembelajaran:

1. Memahami konsep logika dan algoritma komputer
2. Membuat algoritma sederhana untuk menyelesaikan permasalahan
3. Menuliskan program sederhana pseudocode yang dekat dengan bahasa komputer

Algoritma Data

Konsep Dasar Algoritma

Pengertian algoritma

Algoritma dalam kehidupan sehari-hari

Notasi Algoritma (Deskripsi, pseudocode, flowchart)

Algoritma Pencarian

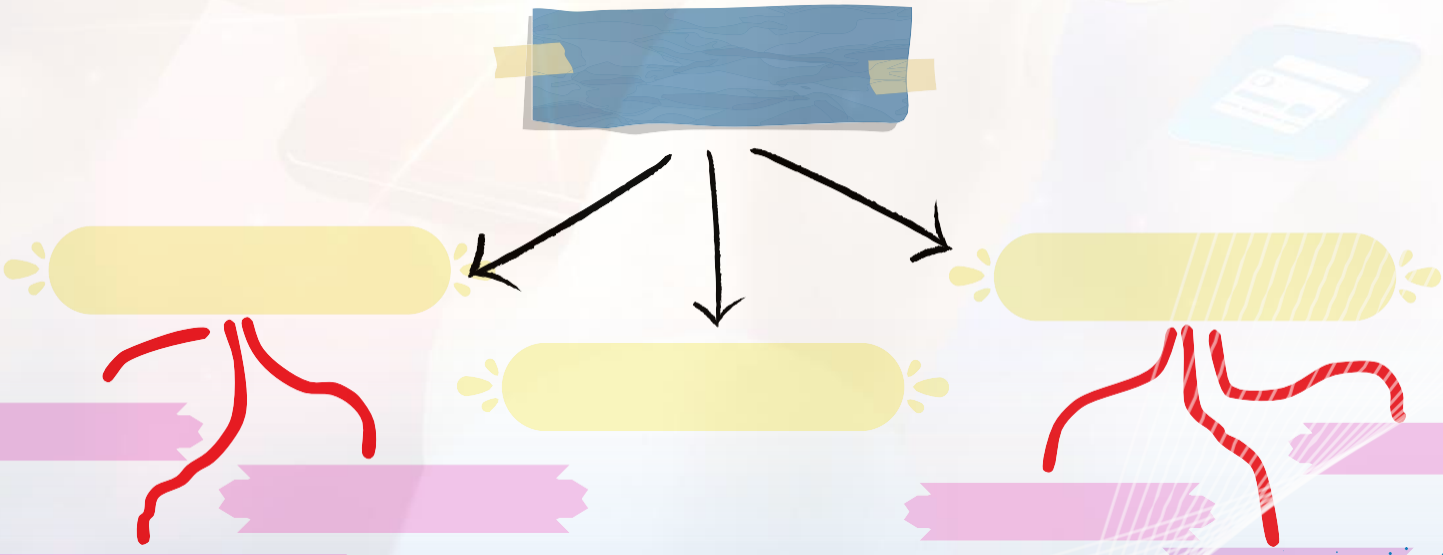
Algoritma Pengurutan

Bubble sort

Insertion sort

selection sort

TOPIK 4
ALGORITMA DATA



A. Konsep Dasar Algoritma

1. Pengertian Algoritma



Apa itu Algoritma?

Algoritma bisa diibaratkan sebagai resep masakan. Sama seperti resep yang memberikan langkah-langkah yang harus diikuti untuk membuat makanan yang enak, algoritma juga memberikan langkah-langkah yang harus diikuti untuk menyelesaikan suatu masalah.

Secara sederhana, **algoritma adalah urutan langkah-langkah logis yang disusun secara sistematis untuk menyelesaikan suatu masalah.**

Mengapa Algoritma Penting?

- Memudahkan pemecahan masalah: Dengan mengikuti langkah-langkah yang jelas, kita dapat menyelesaikan masalah dengan lebih efektif.
- Menjamin hasil yang konsisten: Jika algoritma diikuti dengan benar, hasil yang diperoleh akan selalu sama.
- Dapat diotomatisasi: Algoritma dapat diimplementasikan dalam program komputer untuk melakukan tugas secara otomatis.

Komponen Dasar Algoritma:

- Input: Data yang dimasukkan ke dalam algoritma untuk diproses.
- Proses: Langkah-langkah yang dilakukan untuk mengolah data input.
- Output: Hasil akhir dari proses pengolahan data.

2. Algoritma dalam kehidupan sehari-hari

Contoh Algoritma dalam Kehidupan Sehari-hari:

1. **Membuat mie instan:**

- Rebus air
- Masukkan mie ke dalam air yang mendidih.
- Tunggu hingga mie matang.
- Angkat mie dan tiriskan.
- Masukkan bumbu.
- Aduk rata.

2. Mencari buku di perpustakaan:

- Pergi ke rak buku.
- Cari rak yang sesuai dengan kategori buku yang dicari.
- Cari judul buku secara alfabetis.
- Ambil buku yang ditemukan.

3. Notasi Algoritma

Notasi algoritma adalah cara kita menuliskan langkah-langkah (instruksi) dalam suatu algoritma agar mudah dipahami oleh manusia dan komputer. Ini seperti resep masakan, tapi dalam bentuk yang lebih formal dan terstruktur.

Mengapa Kita Membutuhkan Notasi Algoritma?

- Komunikasi: Agar programmer dapat saling memahami dan bekerja sama dalam mengembangkan perangkat lunak.
- Dokumentasi: Sebagai catatan mengenai bagaimana sebuah program bekerja.
- Perencanaan: Membantu dalam merancang algoritma sebelum diimplementasikan dalam bahasa pemrograman.

3.1 Jenis-Jenis Notasi Algoritma

a. Notasi Deskriptif

Ciri-ciri Notasi Deskriptif:

- Mudah dipahami: Menggunakan bahasa sehari-hari yang mudah dimengerti oleh siapa saja.
- Kurang formal: Tidak memiliki struktur yang baku seperti pseudocode atau flowchart.
- Kurang presisi: Karena menggunakan bahasa sehari-hari, notasi deskriptif bisa menjadi kurang presisi dan sulit untuk diterjemahkan ke dalam bahasa pemrograman.

Kapan Menggunakan Notasi Deskriptif?

- Tahap awal perencanaan: Saat kita baru mulai merancang algoritma, notasi deskriptif bisa membantu kita untuk memvisualisasikan langkah-langkahnya secara sederhana.
- Komunikasi dengan non-teknis: Jika kita ingin menjelaskan algoritma kepada orang yang tidak memiliki latar belakang komputer, notasi deskriptif lebih mudah dipahami.

Contoh 1: Menghitung luas persegi panjang

1. Masukkan nilai panjang persegi panjang.
2. Masukkan nilai lebar persegi panjang.
3. Kalikan nilai panjang dengan nilai lebar.
4. Hasil perkalian adalah luas persegi panjang

Contoh 2: Menghitung luas persegi panjang

1. Masukkan nilai panjang persegi panjang.
2. Masukkan nilai lebar persegi panjang.
3. Kalikan nilai panjang dengan nilai lebar.
4. Hasil perkalian adalah luas persegi panjang.

b. Flowchart/Diagram Alir

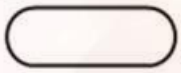
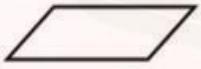
Bayangkan kamu sedang membuat kue. Setiap langkah pembuatan kue, mulai dari menyiapkan bahan hingga kue matang, dituliskan dalam sebuah diagram yang mudah dipahami. Diagram inilah yang disebut flowchart.

Dalam dunia pemrograman, flowchart adalah diagram alir yang menggambarkan langkah-langkah penyelesaian suatu masalah secara visual. Setiap langkah dalam algoritma direpresentasikan dengan simbol-simbol tertentu, dan dihubungkan dengan garis panah untuk menunjukkan urutan pelaksanaannya.

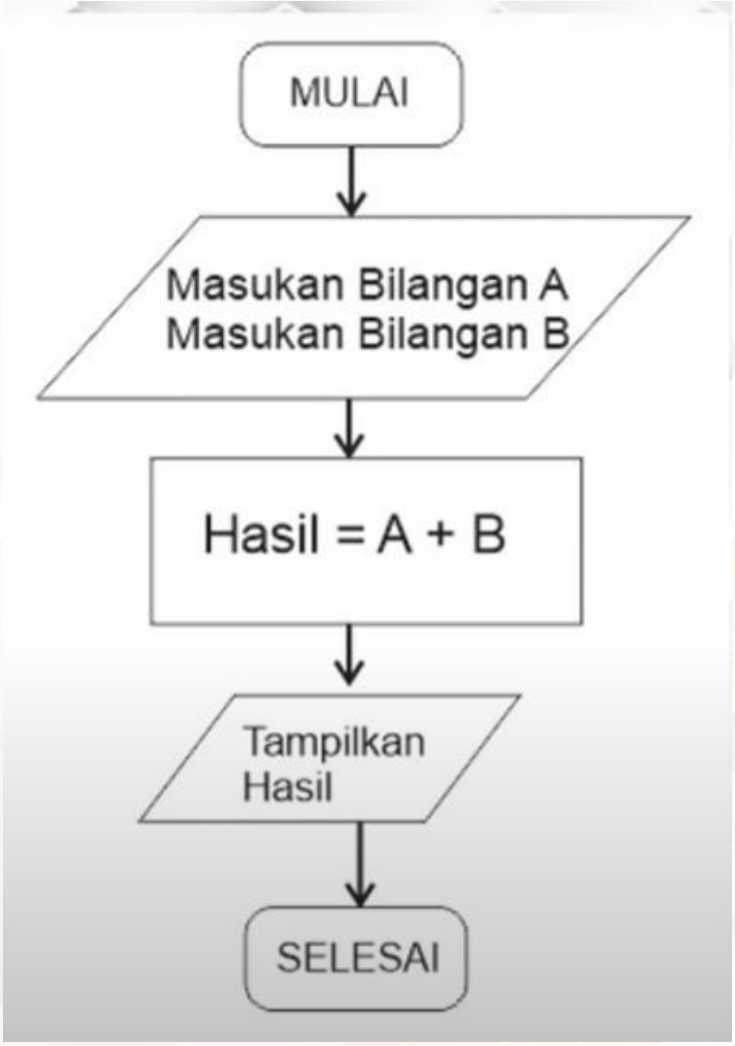
Mengapa Menggunakan Flowchart?

- Memudahkan Visualisasi: Flowchart membuat algoritma lebih mudah dipahami, terutama untuk masalah yang kompleks.
- Perencanaan: Sebelum menulis kode program, membuat flowchart membantu dalam merancang langkah-langkah yang logis dan efisien.

Simbol FlowChart

SIMBOL	NAMA	KETERANGAN
	Terminator	Menyatakan titik awal dan titik akhir dalam suatu program
	*Input / Output*	Menyatakan proses masukan dan proses keluaran
	Proses	Menunjukkan proses yang sedang berjalan atau proses yang sedang dilakukan
	Pencabangan	Menunjukkan kondisi yang akan menghasilkan dua kemungkinan (ya/tidak)

Contoh Flowchart Algoritma Penjumlahan Bilangan



Contoh Flowchart Algoritma Pembuatan Kopi



c. Pseudocode

Pseudocode adalah cara kita menuliskan algoritma (langkah-langkah untuk menyelesaikan suatu masalah) menggunakan bahasa yang mirip dengan bahasa pemrograman, tapi lebih sederhana dan tidak terikat pada aturan sintaks yang ketat. Bayangkan pseudocode seperti resep masakan, tapi ditulis dalam bahasa yang lebih teknis.

Mengapa Kita Menggunakan Pseudocode?

- Perencanaan: Sebelum membuat program sebenarnya, kita bisa menuliskan algoritma dalam bentuk pseudocode untuk merencanakan langkah-langkahnya.
- Komunikasi: Pseudocode lebih mudah dipahami dibandingkan kode program yang kompleks, sehingga bisa digunakan untuk berkomunikasi dengan programmer lain atau menjelaskan algoritma kepada orang yang tidak terlalu paham pemrograman.

Ciri-ciri Pseudocode

- Tidak Terikat Sintaks: Tidak ada aturan sintaks yang ketat seperti dalam bahasa pemrograman.
- Mudah Dipahami: Bahasa yang digunakan cenderung lebih sederhana dan mudah dipahami.
- Fokus pada Logika: Lebih menekankan pada urutan langkah-langkah dan logika algoritma, bukan pada detail sintaks.

Kata Kunci Umum dalam Pseudocode:

- MULA, AKHIR: Menandai awal dan akhir dari suatu blok kode.
- JIKA, MAKA, LAINNYA: Untuk membuat keputusan.
- UNTUK, SAMPAI: Untuk perulangan.
- BACA, TAMPILKAN: Untuk input dan output data.

Contoh 1: Menghitung Luas Segitiga

```
MULAI
    BACA alas, tinggi
    luas <- (alas * tinggi) / 2
    TAMPILKAN luas
AKHIR
```

Penjelasan:

1. MULAI: Menandai awal dari algoritma.
2. BACA alas, tinggi: Meminta pengguna untuk memasukkan nilai alas dan tinggi segitiga.
3. luas <- (alas * tinggi) / 2: Menghitung luas segitiga menggunakan rumus luas segitiga (luas = alas * tinggi / 2) dan menyimpan hasilnya dalam variabel luas.
4. TAMPILKAN luas: Menampilkan hasil perhitungan luas ke layar.
5. AKHIR: Menandai akhir dari algoritma.

Penjelasan Sederhana:

- Alas dan tinggi adalah dua sisi segitiga yang tegak lurus.
- Luas adalah hasil perhitungan luas segitiga.
- Tanda <- artinya "diberi nilai". Jadi, luas <- (alas * tinggi) / 2 artinya nilai dari perhitungan di sebelah kanan tanda sama dengan diberikan kepada variabel luas.

Contoh 1: Menghitung Luas Segitiga

```
MULAI
    BACA bilangan1, bilangan2
    JIKA bilangan1 > bilangan2 MAKA
        TAMPILKAN bilangan1
    LAINNYA
        TAMPILKAN bilangan2
    AKHIR JIKA
AKHIR
```

Penjelasan Langkah demi Langkah:

1. MULA: Menandai awal dari proses atau algoritma yang akan kita lakukan.
2. BACA bilangan1, bilangan2: Ini adalah instruksi untuk meminta pengguna memasukkan dua buah bilangan. Bilangan-bilangan ini akan disimpan dalam variabel bilangan1 dan bilangan2.
3. JIKA bilangan1 > bilangan2 MAKA: Ini adalah sebuah kondisi. Program akan memeriksa apakah nilai yang disimpan dalam bilangan1 lebih besar dari nilai yang disimpan dalam bilangan2.
4. TAMPILKAN bilangan1: Jika kondisi pada langkah 3 benar (artinya bilangan1 memang lebih besar), maka program akan menampilkan nilai bilangan1 di layar. Ini berarti bilangan1 adalah bilangan terbesar.
5. LAINNYA: Jika kondisi pada langkah 3 salah (artinya bilangan1 tidak lebih besar dari bilangan2), maka program akan melanjutkan ke langkah ini.

6. TAMPILKAN bilangan2: Karena bilangan1 tidak lebih besar, maka secara otomatis bilangan2 adalah bilangan terbesar. Jadi, program akan menampilkan nilai bilangan2 di layar.
7. AKHIR JIKA: Menandai akhir dari kondisi JIKA.
8. AKHIR: Menandai akhir dari seluruh proses.

Ilustrasi Sederhana:

Misalkan kita memasukkan nilai 15 untuk bilangan1 dan 8 untuk bilangan2. Program akan menjalankan langkah-langkah sebagai berikut:

1. Program menyimpan 15 di bilangan1 dan 8 di bilangan2.
2. Program memeriksa apakah 15 > 8. Karena benar, maka program akan langsung ke langkah 4.
3. Program menampilkan 15 sebagai output (karena 15 adalah bilangan terbesar).

Konsep Penting:

- Kondisi: JIKA ... MAKA ... LAINNYA adalah struktur dasar dalam pemrograman untuk membuat keputusan. Program akan mengevaluasi kondisi (dalam hal ini, apakah bilangan1 lebih besar dari bilangan2) dan menjalankan blok kode yang sesuai.
- Variabel: bilangan1 dan bilangan2 adalah contoh variabel. Variabel digunakan untuk menyimpan data sementara selama program berjalan.
- Input dan Output: BACA digunakan untuk menerima input dari pengguna, sedangkan TAMPILKAN digunakan untuk menampilkan output ke pengguna.

Pseudocode ini sangat sederhana namun menggambarkan konsep dasar dalam pemrograman:

- Membaca input: Meminta data dari pengguna.
- Membuat keputusan: Membandingkan nilai dan memilih tindakan berdasarkan hasil perbandingan.
- Menampilkan output: Menunjukkan hasil perhitungan atau keputusan program.

B. Algoritma Pencarian

1. Pengertian Algoritma Pencarian



Algoritma pencarian adalah kumpulan langkah-langkah sistematis yang digunakan untuk menemukan data tertentu dalam suatu kumpulan data yang lebih besar. Bayangkan kamu memiliki buku telepon yang sangat tebal dan ingin mencari nomor telepon temanmu. Algoritma pencarian adalah seperti petunjuk langkah demi langkah yang akan kamu ikuti untuk menemukan nomor telepon tersebut dengan cepat.

Mengapa Algoritma Pencarian Penting?

Dalam ilmu komputer, kita seringkali berurusan dengan kumpulan data yang sangat besar. Misalnya, sebuah mesin pencari harus mencari jutaan halaman web untuk menemukan halaman yang sesuai dengan kata kunci yang kamu masukkan. Algoritma pencarian memungkinkan komputer untuk melakukan pencarian ini dengan cepat dan efisien.

2. Jenis-Jenis algoritma Pencarian

a. **Pencarian Linier (Linear Search)**

Algoritma ini memeriksa setiap elemen dalam daftar secara berurutan hingga data yang dicari ditemukan. Ini adalah algoritma yang paling sederhana, tetapi juga paling lambat, terutama untuk daftar yang besar.

Contoh Pencarian Linier (Linear Search) dalam kehidupan sehari-hari:

- Mencari Buku di Rak: Bayangkan kamu memiliki rak buku yang berisi banyak buku dengan judul yang berbeda-beda. Untuk menemukan buku tertentu, kamu akan memeriksa judul setiap buku satu per satu mulai dari buku pertama hingga buku terakhir. Ini adalah contoh sederhana dari pencarian linier.

Contoh Pencarian Linier (Linear Search) dalam Pemrograman

- Mencari Data dalam Database Sederhana: Jika kamu memiliki database sederhana yang menyimpan data siswa dalam bentuk daftar, dan ingin mencari data siswa berdasarkan nomor induk siswa, kamu bisa menggunakan pencarian linier untuk memeriksa setiap record dalam database hingga menemukan NIS yang sesuai.

Implementasi Sederhana dalam Pseudocode

```
MULAI
  BACA daftar_data, data_yang_dicari
  UNTUK setiap elemen dalam daftar_data LAKUKAN
    JIKA elemen = data_yang_dicari MAKA
          TAMPILKAN "Data ditemukan pada indeks ke-",
indeks_elemen
        KELUAR DARI PERULANGAN
    AKHIR JIKA
  AKHIR UNTUK
  JIKA tidak ditemukan MAKA
    TAMPILKAN "Data tidak ditemukan"
AKHIR
```

Pencarian Biner (Binary Search)

Algoritma ini adalah metode pencarian yang sangat efisien untuk menemukan suatu elemen dalam sebuah daftar yang telah terurut. Prinsip kerjanya adalah dengan membagi daftar menjadi dua bagian secara berulang, kemudian membandingkan elemen tengah dengan nilai yang dicari. Jika nilai yang dicari lebih kecil dari elemen tengah, maka pencarian dilanjutkan pada bagian kiri; jika lebih besar, maka pencarian dilanjutkan pada bagian kanan. Proses ini berulang hingga nilai ditemukan atau daftar tidak dapat dibagi lagi.

Contoh Biner (Binary Search) dalam kehidupan sehari-hari:

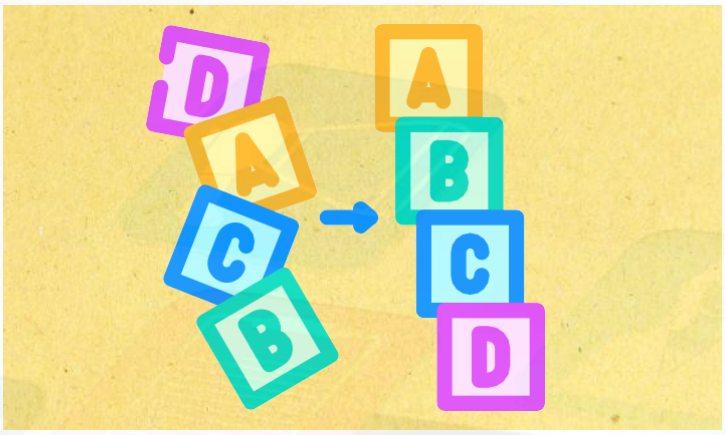
- Mencari Kata dalam Kamus: Bayangkan kamuingin mencari kata "komputer" dalam kamus. Kamu tidak akan membuka kamus dari halaman pertama, lalu halaman kedua, dan seterusnya. Sebaliknya, kamu akan membuka kamus di sekitar bagian tengah. Jika kata "komputer" berada di halaman yang lebih belakang, kamu akan membuang bagian depan kamus dan melanjutkan pencarian di bagian belakang. Proses ini berulang hingga kamu menemukan kata "komputer" atau mencapai akhir kamus.
- Mencari File di Folder: Bayangkan Anda memiliki folder yang berisi banyak file dengan nama yang sudah diurutkan secara alfabetis. Anda ingin mencari file bernama "gambar_presentasi.pptx".

Cara Kerja Pencarian Biner:

1. Menentukan Batas:
    - Batas awal (left): File pertama di folder.
    - Batas akhir (right): File terakhir di folder.

C. Algoritma Pengurutan (Sorting)

1. Pengertian Algoritma Pengurutan)Sorting)



ayangkan kamu punya banyak buku yang berantakan di rak. Untuk memudahkan mencari buku tertentu, kamu akan mengurutkan buku-buku tersebut berdasarkan abjad. Nah, proses mengurutkan buku-buku itu secara sistematis disebut dengan sorting. Dalam dunia komputer, algoritma sorting digunakan untuk mengurutkan data, seperti angka, kata, atau objek, berdasarkan aturan tertentu. Misalnya, mengurutkan angka dari yang terkecil ke terbesar, atau mengurutkan kata-kata berdasarkan abjad. Algoritma sorting adalah suatu metode atau urutan langkah-langkah yang digunakan untuk mengurutkan suatu kumpulan data (misalnya, angka, kata, objek) berdasarkan kriteria tertentu, seperti dari yang terkecil ke terbesar, dari yang terbesar ke terkecil, atau berdasarkan abjad. Dengan memahami konsep ini, kamu akan lebih mudah memahami bagaimana komputer mengolah data dan menyelesaikan masalah.

Mengapa Sorting Penting?

- Memudahkan pencarian: Data yang sudah terurut akan jauh lebih mudah dicari. Bayangkan mencari nama teman di daftar telepon yang sudah terurut alfabet, pasti lebih cepat dibandingkan jika daftarnya acak.
- Meningkatkan efisiensi: Banyak algoritma lain yang memerlukan data yang sudah terurut untuk bekerja dengan baik.
- Memudahkan analisis data: Data yang terurut akan lebih mudah dianalisis dan divisualisasikan.

2. Jenis-Jenis algoritma Pengurutan)Sorting)

a. Insertion Sort

Bayangkan kamu sedang mengurutkan kartu remi di tanganmu. Kamu mulai dengan mengambil satu kartu pertama, lalu mengambil kartu kedua dan membandingkannya dengan kartu pertama. Jika kartu kedua lebih kecil, kamu tukar posisinya. Kemudian, kamu mengambil kartu ketiga dan membandingkannya dengan dua kartu sebelumnya, lalu menempatkannya pada posisi yang benar. Proses ini berulang hingga semua kartu terurut.

Cara Kerja Insertion Sort:

1. Anggap elemen pertama sudah terurut.
2. Ambil elemen kedua.
3. Bandingkan elemen kedua dengan elemen-elemen di sebelah kirinya yang sudah terurut.
4. Geser elemen-elemen yang lebih besar satu posisi ke kanan hingga menemukan posisi yang tepat untuk elemen kedua.
5. Ulangi langkah 2-4 untuk semua elemen yang belum terurut.

Contoh:

Misalkan kita punya array [5, 2, 4, 6, 1, 3].

- Iterasi 1: Elemen pertama (5) dianggap sudah terurut.
- Iterasi 2: Ambil elemen kedua (2). Karena 2 < 5, kita tukar posisinya. Array menjadi [2, 5, 4, 6, 1, 3].
- Iterasi 3: Ambil elemen ketiga (4). Karena 4 > 2, tapi 4 < 5, kita tidak perlu menukar.
- Iterasi 4: Ambil elemen keempat (6). Karena 6 > 2 dan 6 > 5, 6 sudah berada di posisi yang benar.

- Iterasi 5: Ambil elemen kelima (1). Karena 1 < 2, 1 < 5, dan 1 < 4, kita geser 2, 5, dan 4 satu posisi ke kanan, lalu tempatkan 1 di awal. Array menjadi [1, 2, 4, 5, 6, 3].
- Iterasi 6: Ambil elemen keenam (3). Karena 3 > 1, 3 > 2, dan 3 < 4, kita geser 4 dan 5 satu posisi ke kanan, lalu tempatkan 3 di antara 2 dan 4. Array menjadi [1, 2, 3, 4, 5, 6].

b.Insertion Sort

Bayangkan kamu punya sekotak kelereng yang berantakan. Kamu ingin mengurutkannya dari yang terkecil ke yang terbesar.

Bagaimana Selection Sort bekerja?

1. Cari kelereng terkecil: Kamu mencari kelereng terkecil di seluruh kotak.
2. Tukar posisi: Kamu mengambil kelereng terkecil itu dan menaruhnya di awal kotak.
3. Ulangi: Kamu mengulangi langkah 1 dan 2 untuk sisa kelereng yang belum terurut, setiap kali mencari kelereng terkecil dari sisa kelereng.

Contoh:

Misal kita punya deretan angka: 64, 25, 12, 22, 11.

- Iterasi 1:
  - Cari angka terkecil: 11.
  - Tukar 11 dengan angka pertama (64).
  - Deretan menjadi: 11, 25, 12, 22, 64.
- Iterasi 2:
  - Dari sisa deretan (25, 12, 22, 64), cari angka terkecil: 12.
  - Tukar 12 dengan angka kedua (25).
  - Deretan menjadi: 11, 12, 25, 22, 64.
- Dan seterusnya...

Proses berulang hingga semua angka terurut.

c. Bubble Sort

Bubble Sort adalah algoritma pengurutan yang sederhana, tapi kurang efisien untuk data yang besar. Elemen yang lebih besar akan secara bertahap "menggelembung" ke akhir daftar. Bayangkan kamu punya segelas air soda yang baru dibuka. Gelembung-gelembung udara akan naik ke atas secara perlahan. Bubble Sort bekerja dengan prinsip yang mirip.

Bagaimana Bubble Sort bekerja?

1. Bandingkan dan tukar: Kita mulai dari awal daftar. Bandingkan dua elemen pertama. Jika elemen pertama lebih besar dari yang kedua, tukar posisinya.
2. Ulangi: Lanjutkan membandingkan dan menukar pasangan elemen berikutnya hingga akhir daftar.
3. Ulangi proses: Ulangi langkah 1 dan 2 sampai tidak ada lagi pertukaran yang terjadi. Artinya, daftar sudah terurut.

Contoh:

Misal kita punya deretan angka: 64, 34, 25, 12, 22.

- Iterasi 1:
  - Bandingkan 64 dan 34, tukar karena 64 > 34.
  - Bandingkan 64 dan 25, tukar.
  - Bandingkan 64 dan 12, tukar.
  - Bandingkan 64 dan 22, tukar.
  - Deretan menjadi: 34, 25, 12, 22, 64.
- Iterasi 2:
  - Ulangi proses perbandingan dan penukaran.
  - ...
- Ulangi terus hingga tidak ada lagi pertukaran.

Kenapa disebut Bubble Sort?

Karena elemen yang lebih besar akan "menggelembung" ke atas seperti gelembung udara dalam air soda.

Kelebihan Bubble Sort:

- Sederhana: Konsepnya mudah dipahami.

Kekurangan Bubble Sort:

- Tidak efisien untuk data yang besar: Jumlah perbandingan yang dilakukan sangat banyak, terutama untuk data yang besar dan tidak terurut.

Kapan Menggunakan Bubble Sort?

- Data berukuran sangat kecil: Untuk data yang sedikit, Bubble Sort mungkin cukup cepat.
- Ketika kemudahan pemahaman lebih diutamakan: Karena algoritmanya sederhana.

Contoh Kasus Penerapan Bubble Sort

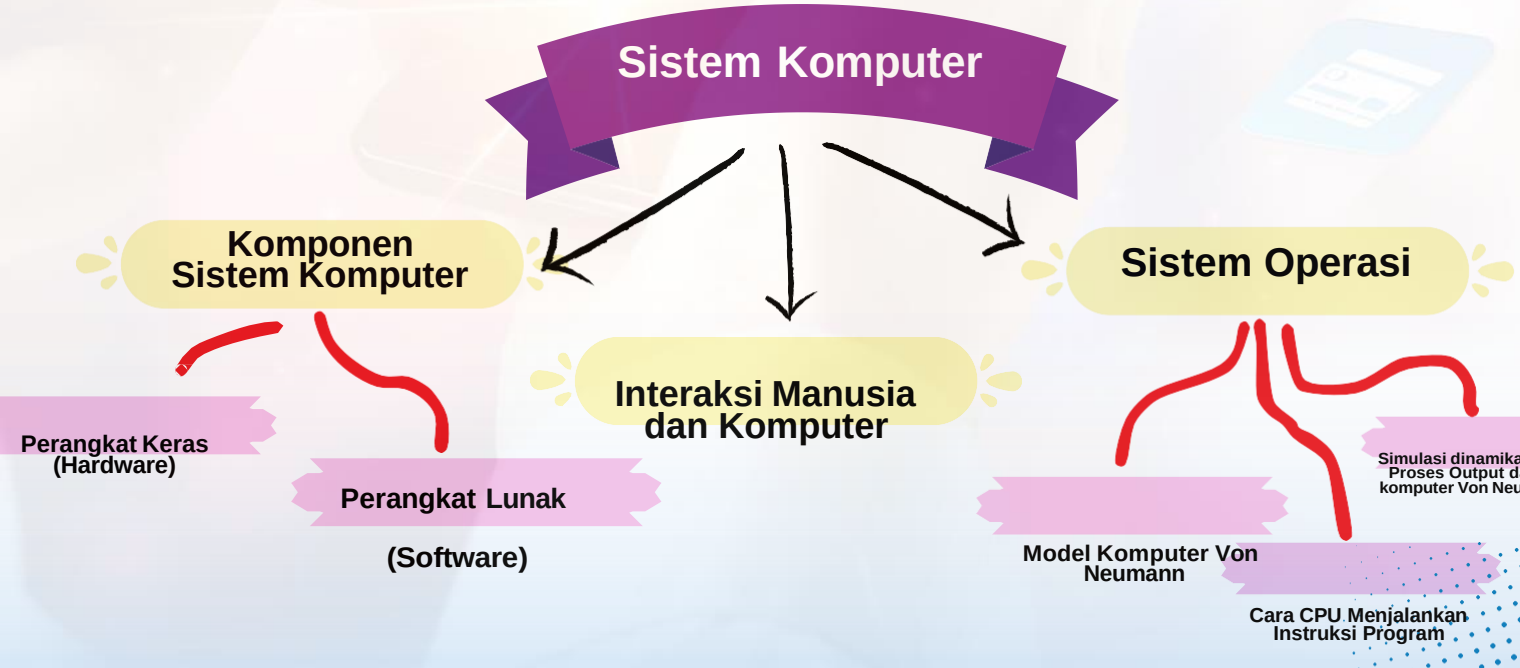
1. Pengurutan Nilai Mahasiswa:
   - Kasus: Anda memiliki daftar nilai mahasiswa dalam sebuah array. Anda ingin mengurutkan nilai-nilai tersebut dari yang tertinggi ke terendah untuk menentukan peringkat.
   - Solusi: Gunakan Bubble Sort untuk membandingkan nilai setiap mahasiswa secara berpasangan. Nilai yang lebih besar akan "digelembungkan" ke atas hingga semua nilai terurut.
2. Pengurutan Data Produk di Toko Online:
   - Kasus: Sebuah toko online memiliki banyak produk dengan harga yang berbeda. Anda ingin menampilkan produk dari yang termurah ke yang termahal.
   - Solusi: Gunakan Bubble Sort untuk mengurutkan data produk berdasarkan harga.
3. Pengurutan Data Kontak:
   - Kasus: Anda memiliki daftar kontak dengan nama-nama yang tidak berurutan. Anda ingin mengurutkan daftar kontak berdasarkan abjad.
   - Solusi: Terapkan Bubble Sort untuk membandingkan nama setiap kontak dan menukarnya jika urutannya salah.

TOPIK 5
SISTEM KOMPUTER



Tujuan Pembelajaran:

1. Memahami komponen penyusun komputer
2. Memahami interaksi antara perangkat keras, perangkat lunak dan pengguna komputer
3. Memahami model dinamika Input Proses Output dalam sebuah komputer Von Neumann
4. Mensimulasikan dinamika Input Proses Output dalam sebuah komputer Von Neumann



A. Pengenalan Sistem Komputer

1. Pengertian Sistem Komputer



Bayangkan komputer seperti sebuah pabrik mini. Pabrik ini punya mesin-mesin (perangkat keras) dan pekerja (perangkat lunak) yang bekerja sama untuk menghasilkan produk (informasi).

Jadi, sistem komputer adalah kumpulan dari perangkat keras dan perangkat lunak yang bekerja sama untuk memproses data dan menghasilkan informasi yang berguna bagi kita.

2. Komponen Sistem Komputer

a. Perangkat Keras (Hardware)



- CPU (Central Processing Unit): Otak komputer, yang menjalankan semua perintah.
- Memori: Tempat penyimpanan data sementara saat komputer sedang bekerja.
- Perangkat Input: Alat untuk memasukkan data ke komputer (misal: keyboard, mouse).
- Perangkat Output: Alat untuk mengeluarkan hasil dari komputer (misal: monitor, printer).
- Penyimpanan: Tempat menyimpan data secara permanen (misal: hard disk, SSD).

b. Perangkat Lunak (Software)

- Sistem Operasi: Program utama yang mengelola semua perangkat keras dan menjalankan program lain (misal: Windows, Linux).
- Aplikasi: Program yang digunakan untuk tugas tertentu (misal: Microsoft Word, Google Chrome).

3. Bagaimana Komputer Bekerja

Bagaimana Sistem Komputer Bekerja?

1. Kamu memberikan perintah: Misalnya, kamu mengetik kata di keyboard.
2. Perangkat input (keyboard) mengirimkan data ke CPU.
3. CPU memproses data sesuai dengan program yang sedang berjalan.
4. Hasilnya ditampilkan di perangkat output (misal: muncul di layar monitor).

Contoh Sederhana:

Ketika kamu membuka dokumen Word dan mengetik sebuah kalimat, inilah yang terjadi di dalam komputer:

- Keyboard: Menerima input berupa huruf dan spasi yang kamu ketik.
- CPU: Menerima input dari keyboard, lalu menyimpannya dalam memori.
- Word: Program Word memproses data yang sudah disimpan di memori dan menampilkannya di layar.

B. Interaksi Manusia dan Komputer

1. Jenis-Jenis Interaksi Manusia dan Komputer

a. **Berbasis GUI (Graphical User Interface):**

- Paling umum digunakan: Hampir semua perangkat elektronik modern menggunakan GUI.
- Ciri: Menggunakan elemen grafis seperti ikon, tombol, menu untuk berinteraksi.
- Contoh: Smartphone, komputer desktop, laptop.

b. Antarmuka Berbasis Perintah (Command Line Interface/CLI):

- Menggunakan teks: Pengguna mengetik perintah secara langsung.
- Lebih fleksibel: Memberikan kontrol penuh kepada pengguna.
- Contoh: Terminal Linux, DOS.

c. Melalui Suara (Audio):

- Penggunaan suara sebagai input: Perintah diberikan secara lisan.
- Contoh: Asisten virtual seperti Siri, Google Assistant.

d. Melalui Gambar atau Video:

- Menggunakan gambar atau video sebagai input: Misalnya, mengenali wajah, objek, atau gerakan.
- Contoh: Aplikasi pengenalan wajah, augmented reality.

e. Melalui Berbagai Piranti Masukan Lainnya:

- Beragam perangkat: Keyboard, mouse, touchpad, joystick, keyboard virtual, layar sentuh, dan lainnya.
- Fungsinya: Memungkinkan pengguna berinteraksi dengan komputer secara lebih natural.

C. Sistem Operasi

Sistem operasi adalah perangkat lunak dasar yang berfungsi sebagai penghubung antara pengguna dan perangkat keras komputer. Bayangkan sistem operasi sebagai manajer yang mengatur semua aktivitas di dalam komputer, mulai dari menjalankan program, mengelola file, hingga mengatur penggunaan memori dan perangkat keras lainnya.

1. Model Komputer Von Neumann

Model Von Neuman (atau Mesin Von neuman) adalah model yang diciptakan oleh John von Neumann (1903-1957). Arsitektur/model ini digunakan oleh hampir semua komputer saat ini. Arsitektur Von Neumann menggambarkan komputer dengan empat bagian utama: Unit Aritmetika dan Logis (ALU), unit kontrol, memori, dan alat masukan dan hasil (secara kolektif dinamakan I/O). Bagian ini dihubungkan oleh berkas kawat, "bus".

John Van Neumann seorang ahli matematika yang merupakan konsultan pembuatan ENIAC pada tahun 1945 mencoba memperbaiki kelemahan ENIAC dengan rancangan komputer barunya, bernama EDVAC (Electronic Discrete Variable Computer) dengan konsep program tersimpan (bahasa Inggris: stored program concept). Tahun 1946 komputer dengan stored-program concept dipublikasikasikan, yang kemudian di kenal dengan Komputer IAS (Computer of Institute for Advanced Studies).

Bayangkan komputer seperti sebuah pabrik kecil.

- Pabrik ini memiliki gudang: Gudang ini adalah memori. Di sinilah semua bahan baku (data) dan instruksi kerja (program) disimpan.
- Pabrik ini memiliki mesin produksi: Mesin ini adalah Unit Aritmatika dan Logika (ALU). Mesin ini bertugas untuk mengolah bahan baku menjadi produk jadi sesuai dengan instruksi.

- Pabrik ini memiliki seorang mandor: Mandor ini adalah Unit Kontrol. Mandor ini yang mengatur semua aktivitas di pabrik, mulai dari mengambil bahan baku dari gudang, menjalankan mesin produksi, hingga mengirimkan produk jadi ke gudang atau ke luar pabrik.

Bagaimana proses kerjanya?

- Penerimaan Pesanan: Kamu memberikan perintah (input) ke komputer, misalnya mengetik sebuah kata di keyboard.
- Pengambilan Bahan Baku: Perintah tersebut disimpan dalam memori sebagai data.
- Pembuatan Produk: Unit kontrol mengambil instruksi dari memori dan memberikannya ke ALU. ALU kemudian mengolah data sesuai instruksi.
- Pengiriman Produk: Hasil olahan (output) ditampilkan di layar monitor.

Contoh Sederhana:

Ketika kamu mengetik huruf "A" di keyboard, komputer akan:

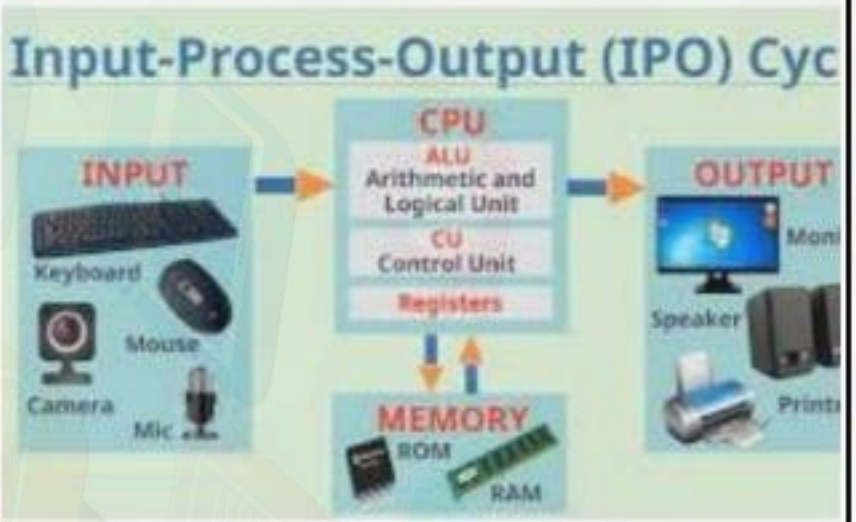
1. Menerima sinyal dari keyboard.
2. Mengubah sinyal tersebut menjadi kode biner yang merepresentasikan huruf "A".
3. Menyimpan kode biner tersebut dalam memori.
4. Mengambil instruksi untuk menampilkan huruf "A" di layar.
5. Menjalankan instruksi tersebut, sehingga huruf "A" muncul di layar.

Jadi, Model Von Neumann adalah cara sederhana untuk memahami bagaimana komputer bekerja. Semua komputer, dari yang paling sederhana hingga yang paling canggih, pada dasarnya mengikuti model ini.



Hardware: Bagian & Organ Tubuh Manusia & Otak

Software: Sistem Peredaran Darah, Pencernaan, Pikiran



Hardware: Perangkat Keras

Software: Operating System Utilities

Gambar 2.18 Analogi Tubuh Manusia dan Sistem Komputer

2. Cara CPU Menjalankan Instruksi Program



CPU (Central Processing Unit) sering disebut sebagai "otak" dari komputer. Komponen ini bertanggung jawab untuk menjalankan semua instruksi yang diberikan kepada komputer. Bayangkan CPU sebagai seorang pekerja pabrik yang sangat cepat dan efisien, yang menerima instruksi, memproses data, dan menghasilkan output.

a. Komponen CPU

1. Unit Aritmatika dan Logika (ALU):
   - Bagian ini melakukan semua perhitungan matematika dan operasi logika seperti penjumlahan, pengurangan, perkalian, pembagian, serta operasi AND, OR, NOT.
   - Misalnya, ketika kamu menghitung 2+2 di kalkulator, ALU-lah yang melakukan perhitungannya.
2. Unit Kontrol:
   - Bagian ini mengatur semua aktivitas di dalam CPU.
   - Ia seperti seorang mandor yang memberi tahu ALU apa yang harus dilakukan, mengambil instruksi dari memori, dan mengkoordinasikan semua operasi.
3. Register:
   - Ini adalah area penyimpanan data sementara yang sangat cepat.
   - Data yang sedang diproses akan disimpan sementara di sini sebelum dikirim ke ALU atau memori utama.

4. Cache:
   - Ini adalah jenis memori yang sangat cepat yang berfungsi sebagai buffer antara CPU dan memori utama.
   - Data yang sering digunakan akan disimpan di cache agar bisa diakses lebih cepat oleh CPU.

Analogi Sederhana

Bayangkan CPU sebagai sebuah dapur.

- ALU adalah kompor dan oven, tempat makanan (data) dimasak (diproses).
- Unit Kontrol adalah koki yang mengatur semua kegiatan di dapur, mulai dari mengambil bahan makanan (data) dari lemari pendingin (memori), hingga menyajikan makanan (hasil) ke meja makan.
- Register adalah talenan dan mangkuk, tempat bahan makanan (data) disiapkan sebelum dimasak.
- Cache adalah lemari pendingin kecil di dekat kompor, tempat bahan makanan yang sering digunakan disimpan agar mudah diambil.

Fungsi Utama CPU

- Mengambil instruksi: Membaca instruksi dari memori.
- Mendekode instruksi: Menerjemahkan instruksi menjadi tindakan yang bisa dilakukan oleh CPU.
- Mengeksekusi instruksi: Melakukan operasi yang diperintahkan oleh instruksi.
- Menyimpan hasil: Menyimpan hasil operasi ke dalam memori.

Singkatnya, CPU adalah jantung dari sebuah komputer. Semua tugas yang kita lakukan di komputer, mulai dari mengetik dokumen hingga bermain game, pada akhirnya akan diproses oleh CPU.

b. Dinamika Proses Input, Proses, Output (IPO)

Bayangkan komputer seperti sebuah pabrik roti.

- Input: Terigu, air, ragi, dan bahan-bahan lainnya masuk ke pabrik (komputer) melalui pintu masuk (perangkat input seperti keyboard, mouse). Ini seperti kamu memasukkan data atau perintah ke komputer.
- Proses: Di dalam pabrik, bahan-bahan tersebut diolah menjadi adonan, dibentuk, dan dipanggang menjadi roti (data diproses oleh CPU). CPU-lah yang menjalankan semua instruksi untuk mengolah data.
- Output: Roti yang sudah jadi keluar dari pabrik melalui pintu keluar (perangkat output seperti monitor, printer). Ini seperti hasil pemrosesan data yang ditampilkan di layar atau dicetak.

Dalam konteks komputer, proses IPO bekerja sebagai berikut:

1. Input: Data dimasukkan ke komputer melalui perangkat input. Data ini bisa berupa angka, teks, gambar, atau suara.
2. Proses: CPU menerima data input, mengolahnya sesuai dengan program yang sedang berjalan, dan menghasilkan output. Proses ini melibatkan:
    - Fetching: Mengambil instruksi dari memori.
    - Decoding: Menerjemahkan instruksi.
    - Executing: Melakukan operasi yang diperintahkan oleh instruksi.
    - Storing: Menyimpan hasil ke memori.
3. Output: Hasil pemrosesan ditampilkan melalui perangkat output.

Contoh Sederhana:

Ketika kamu mengetik kata "halo" di keyboard:

1. Input: Huruf-huruf "h", "a", "l", dan "o" dimasukkan sebagai data.
2. Proses: CPU mengubah huruf-huruf tersebut menjadi kode biner yang bisa dimengerti oleh komputer, kemudian menyimpannya dalam memori.
3. Output: Kata "halo" ditampilkan di layar monitor.

c. Penjelasan Detail Siklus Proses Input, Proses, Output (IPO) di CPU

Siklus instruksi ini berulang terus-menerus selama komputer beroperasi. Setiap instruksi yang diberikan pada komputer akan melalui siklus ini. Berikut adalah tahapan-tahapannya:

1. Fetch (Pengambilan):
   - CPU mengambil instruksi berikutnya dari memori. Instruksi ini berupa kode biner yang mendefinisikan operasi yang akan dilakukan.
   - Alamat memori tempat instruksi berada disimpan dalam program counter (PC). Setelah instruksi diambil, nilai PC akan bertambah sehingga mengarah ke instruksi berikutnya.
2. Decode (Penguraian):
   - Instruksi yang diambil kemudian diuraikan oleh unit kontrol. Unit kontrol akan menerjemahkan kode biner menjadi serangkaian sinyal yang dapat dipahami oleh komponen-komponen lain dalam CPU, seperti ALU dan register.
   - Unit kontrol akan menentukan operasi apa yang harus dilakukan (misalnya, penjumlahan, perbandingan, pemindahan data), serta data apa yang akan digunakan dalam operasi tersebut.

3. Execute (Eksekusi):
    - Instruksi yang sudah diuraikan kemudian dieksekusi. ALU akan melakukan operasi yang telah ditentukan, seperti perhitungan aritmatika atau logika. Hasil dari operasi ini akan disimpan dalam register.
4. Write back (Menulis Kembali):
    - Hasil dari eksekusi instruksi akan ditulis kembali ke memori atau register.

Contoh Sederhana:
Misalkan kita ingin menjumlahkan dua angka, 5 dan 3. Siklus instruksi akan berjalan sebagai berikut:

1. Fetch: CPU mengambil instruksi penjumlahan dari memori.
2. Decode: Instruksi diterjemahkan, dan CPU mengetahui bahwa ia harus menjumlahkan dua angka.
3. Execute: ALU melakukan penjumlahan antara 5 dan 3.
4. Write back: Hasil penjumlahan (8) disimpan dalam register.

Faktor yang Mempengaruhi Kecepatan Siklus Instruksi:

- Frekuensi CPU: Semakin tinggi frekuensi, semakin banyak siklus yang dapat dilakukan dalam satu detik.
- Arsitektur CPU: Desain internal CPU, seperti jumlah core, ukuran cache, dan pipeline, juga mempengaruhi kecepatan.
- Kompleksitas Instruksi: Instruksi yang lebih kompleks membutuhkan waktu eksekusi yang lebih lama.

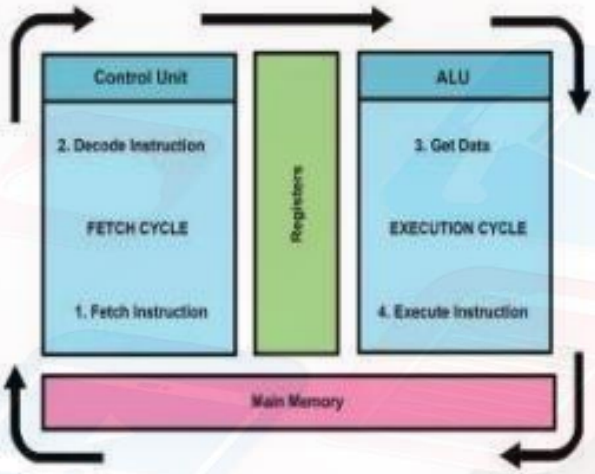
3. Simulasi dinamika Input Proses Output dalam komputer Von Neumann

Tahukah kalian, bahwa kita bisa membuat abstraksi sebuah mesin komputer dengan menciptakan sebuah mesin konseptual sederhana? Kalian diberi sebuah mesin konseptual sederhana ciptaan Mr. ALGO dan menyimulasikan cara kerjanya, yang seperti cara kerja sebuah komputer, tetapi dengan lebih sederhana.

Pada bagian ini, kalian akan memahami bahwa instruksi program dalam bahasa yang lebih dekat ke manusia harus diterjemahkan menjadi instruksi dalam bahasa mesin untuk dapat dijalankan. Ingat bahwa dengan struktur komputer yang terdiri atas input, output, memori dan CPU, komputer hanya dapat membaca data dari perangkat masukan, menulis data ke perangkat keluaran, menyalin data dari CPU ke memori atau dari memori ke CPU, dan melakukan perhitungan aritmatika dan logika. Dengan kemampuan tersebut, kita dapat menuliskan program komputer yang beragam dan luar biasa daya gunanya!

Program komputer terdiri atas sekumpulan instruksi. Instruksi yang dijalankan oleh mesin harus dikenali oleh CPU yang disebut bahasa mesin. Bagaimana sebuah komputer menjalankan sebuah program dalam bahasa mesin? Komputer akan menjalankan (mengeksekusi) perhitungan dengan langkah yang disebut "fetch execute cycle" (siklus ambil dan jalankan). Fetch execute cycle ialah operasi yang paling mendasar dalam komputer, yang juga disebut "fetch decode execute cycle". Selama fetch execute cycle, mesin komputer akan mengambil instruksi dari memori dan menjalankan instruksi tersebut sesuai dengan jenis instruksinya.

Mengapa disebut fetch execute cycle? Siklus untuk fetching, decoding, dan executing sebuah instruksi ini akan diulang-ulang oleh CPU selama mesin komputer hidup!



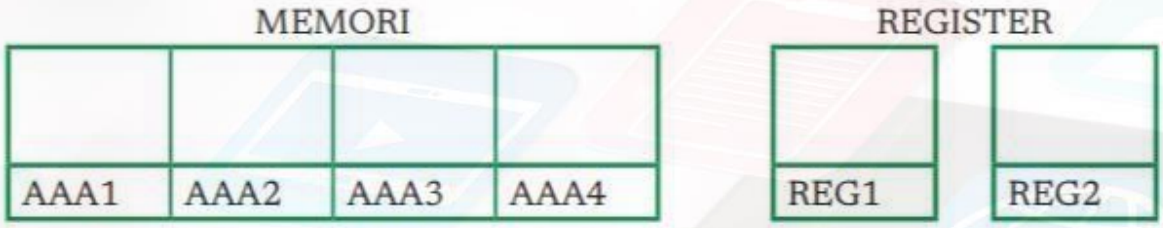
Gambar 4.20 Siklus Ambil dan Jalankan (Fetch Execute Cycle)

Sumber: Dokumen Kemendikbud, 2021

Gambaran siklusnya ditunjukkan dalam Gambar 4.20 yang menunjukkan lebih detail bagian-bagian dari CPU.

1. CU akan mengambil instruksi (Fetch).
2. CU akan menerjemahkan instruksi tersebut harus melakukan apa, misalnya menyimpan data, menghitung, atau lainnya. Ingat, bahwa komputer hanya dapat melakukan perhitungan dan menyimpan/mengambil, atau mengirimkan data ke input/output device.
3. ALU akan mengambil data yang diperlukan untuk menjalankan instruksi, dan data yang sedang diproses disimpan dalam Register.
4. ALU menjalankan instruksi. Berikut ini gambaran Memori dan Register dari sebuah mesin ciptaan Mr. ALGO tersebut. Alamat pada mesin ini dinyatakan dalam kode Heksadesimal. Memori dibagi-bagi menjadi kotak-kotak. Setiap kotak mempunyai alamat dan dapat berisi data.

Misalnya, mesin ciptaan kalian memiliki kapasitas memori untuk menampung 4 data dengan alamat AAA1 s.d AAA4 dan CPU mempunyai 2 register dengan alamat REG1 dan REG2 yang ilustrasikan seperti pada gambar di bawah ini. Sebetulnya, semua data akan disimpan dalam bentuk biner, seperti yang telah dipelajari di jenjang SMP. Namun demikian, untuk kemudahan membaca, data semua ilustrasi, data tetap dituliskan dalam besaran desimal



Sekarang, kalian akan mensimulasi *Fetch instruction cycle* tersebut untuk menjalankan instruksi perhitungan aritmatika sederhana sebagai berikut yang semula ditulis dalam bahasa yang lebih dipahami manusia (bahasa tingkat tinggi).

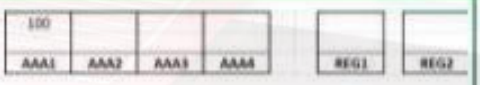
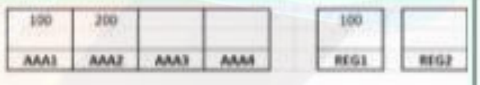
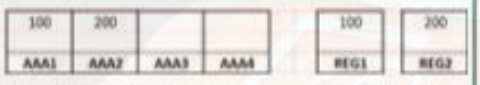
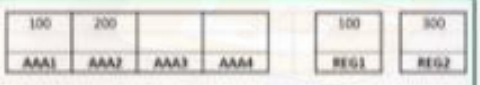
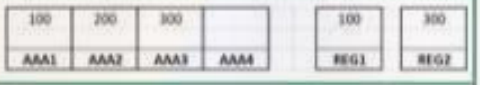
```
X=200
Y=100
Jumlah= X + Y
PRINT Jumlah
```

Mesin Konseptual Sederhana akan mengeksekusi perintah tersebut dalam beberapa langkah karena data disimpan dalam memori (disimpan dalam variabel X dan Y), sedangkan proses perhitungan penjumlahan harus dilakukan oleh ALU yang merupakan bagian dari CPU. Misalnya, nilai dari variabel X disimpan dalam alamat AAA1 dan nilai dari variabel Y disimpan dalam AAA2, dan nilai variabel Jumlah akan disimpan dalam alamat AAA4. Data harus dibawa ke Register untuk dijumlahkan.

Mesin Konseptual Sederhana menjalankan beberapa instruksi bahasa mesin sebagai berikut untuk menjalankan program di atas.

```
SIMPAN 100 AAA1
SIMPAN 200 AAA2
SALIN AAA1 REG1
SALIN AAA2 REG2
TAMBAH REG1 REG2
SALIN REG2 AAA3
PRINT AAA3
```

Eksekusi dari perintah-perintah tersebut secara berturut-turut ditunjukkan dengan ilustrasi sebagai berikut.

Instruksi	Penjelasan	Peta Memori dan CPU
SIMPAN 100 AAA1	Menyimpan data 100 ke dalam memori dengan alamat AAA1.	
SIMPAN 200 AAA2	Menyimpan data 200 ke dalam memori dengan alamat AAA2.	
SALIN AAA1 REG1	Menyalin data dari memori dengan alamat AAA1 ke register dengan alamat REG1.	
SALIN AAA2 REG2	Menyalin data dari memori dengan alamat AAA2 ke register dengan alamat REG2.	
TAMBAH REG1 REG2	Menambahkan data pada alamat REG1 dan data pada alamat REG2 dan mesin akan menyimpan hasilnya pada alamat REG2. *) Baca Penjelasan di bawah tabel.	
SALIN REG2 AAA3	Menyalin data dari register REG2 ke memori AAA3.	
PRINT AAA3	Mencetak data yang ada pada memori AAA3 ke layar monitor mesin untuk ditampilkan.	300

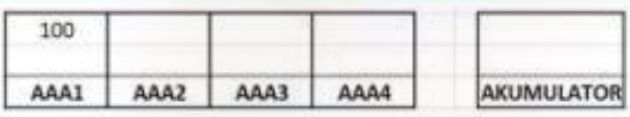
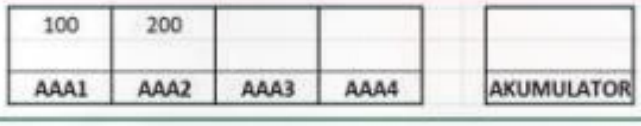
Mesin Konseptual Sederhana tersebut hanya mempunyai 2 register (REG1 dan REG2), dan dirancang agar jika melakukan operasi aritmatika dua buah bilangan, hasilnya disimpan pada register yang menyimpan operan ke-2.

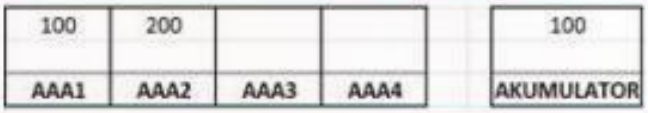
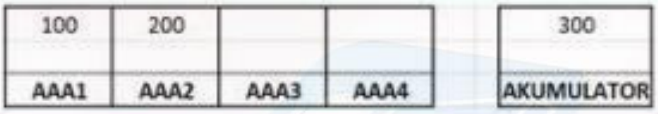
Mungkin saja, ada orang yang menciptakan mesin konseptual lain yang mempunyai 3 register. Seandainya, mesin mempunyai 3 register, bisa saja hasil penjumlahan disimpan pada register ke-3. Jumlah register dan bagaimana operasi dilakukan, serta hasil operasi aritmatika disimpan, itu ditentukan oleh penciptanya, merupakan spesifikasi mesin.

Kita juga dapat menciptakan mesin konseptual yang sangat sederhana, yang hanya mempunyai 1 register untuk menampung data, yang biasanya disebut AKUMULATOR karena semua operan dan hasil perhitungan disimpan di register tunggal. Perintah TAMBAH dilakukan dengan menambahkan data yang diambil dari memori ke satu-satunya register itu. Perintah Mesin misalnya:

INSTRUKSI	HASIL
LOAD <alamat>	Data yang disimpan pada alamat memori disimpan ke AKUMULATOR.
TAMBAH <alamat>	Data yang disimpan pada AKUMULATOR ditambah dengan data yang diambil dari alamat memori.
KURANG <alamat>	Data yang disimpan pada AKUMULATOR dikurangi dengan data yang diambil dari alamat memori.
KALI <alamat>	Data yang disimpan pada pada AKUMULATOR dikalikan dengan data yang diambil dari alamat memori.
BAGI <alamat>	Data yang disimpan pada AKUMULATOR dikurangi dengan data yang diambil dari alamat memori.
OUTPUT	Data AKUMULATOR dikirim ke perangkat keluaran.
INPUT <data><alamat>	Alamat diisi dengan data.

Untuk menambahkan 100+200, instruksi yang dilakukan oleh mesin dengan satu AKUMULATOR yang hanya mempunyai 4 alamat memori AAA1, AAA2, AAA3 dan AAA4 diberikan sebagai berikut.

INSTRUKSI	ISI MEMORI dan CPU (Akumulator)
INPUT 100 AAA1	
INPUT 200 AAA2	

INSTRUKSI	ISI MEMORI dan CPU (Akumulator)
LOAD AAA1	
TAMBAH AAA2	
OUTPUT	300

Jika kalian perhatikan, instruksi-instruksi di atas merupakan rangkaian perintah yang dilakukan untuk menghitung penjumlahan 100+200. Instruksi yang diberikan bergantung pada spesifikasi mesinnya, dan mesin juga dapat mempunyai satu atau lebih register. Pencipta mesin yang akan merancang mesin-mesin komputer tersebut, dan mewujudkannya menjadi perangkat elektronik yang dapat berfungsi sesuai spesifikasi. Tentu saja, gambaran pelaksanaan instruksi di atas ialah penyederhanaan karena yang dilakukan oleh komputer sebenarnya jauh lebih rinci. Data 100 dan 200 dalam desimal pun akan direpresentasi dalam bentuk biner seperti yang kalian telah pelajari di tingkat SMP.

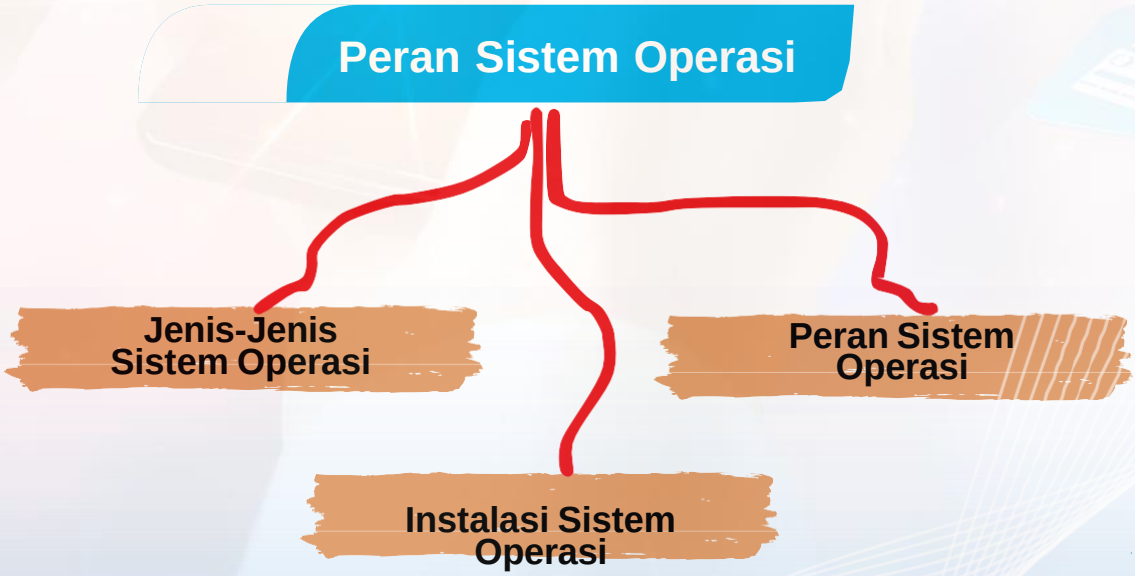
Unit pembelajaran ini memberikan gambaran bahwa ekspresi yang ditulis dalam bahasa yang dimengerti manusia harus diterjemahkan sangat detail agar dapat dieksekusi oleh mesin ciptaan kalian. Komputer yang selama ini kalian gunakan sebenarnya mirip dengan mesin konseptual sederhana tersebut, hanya kecepatan melaksanakan instruksi sangat cepat sekali.

TOPIK 6
PERAN SISTEM OPERASI



Tujuan Pembelajaran:

1. Mengidentifikasi instalasi sistem operasi dan aplikasi pada sebuah perangkat
2. Memahami peran sistem operasi



A. Jenis-Jenis Sistem Operasi

1. Sistem Operasi Desktop

a. Windows:

- Sistem operasi paling populer untuk PC dan laptop.
- Antarmuka pengguna yang intuitif dan mudah digunakan.
- Kompatibel dengan berbagai perangkat keras dan software.
- Varian: Windows 10, Windows 11.
- Kelebihan: Populer, banyak aplikasi, dukungan yang luas.
- Kekurangan: Rentan terhadap virus dan malware, kadang-kadang terasa berat.

b. macOS:

- Sistem operasi eksklusif untuk perangkat Apple (Macbook, iMac).
- Desain yang elegan dan kinerja yang baik.
- Integrasi yang erat dengan perangkat keras Apple.
- Kelebihan: Kinerja tinggi, ekosistem yang terintegrasi, desain yang indah.
- Kekurangan: Harga perangkat yang relatif mahal, pilihan aplikasi terbatas dibandingkan Windows.

c. Linux:

- Sistem operasi open-source yang sangat fleksibel dan customizable.
- Terdapat banyak distribusi Linux (distro) seperti Ubuntu, Fedora, Debian, CentOS.
- Kelebihan: Gratis, sumber terbuka, keamanan tinggi, kinerja yang baik.
- Kekurangan: Kurva pembelajaran yang lebih curam, tidak semua aplikasi tersedia secara native.

2. Sistem Operasi Mobile

a. Android:

- Sistem operasi open-source berbasis Linux yang mendominasi pasar smartphone.
- Fleksibel, banyak pilihan aplikasi, ekosistem yang besar.
- Kelebihan: Terbuka, banyak pilihan perangkat, aplikasi yang beragam.
- Kekurangan: Fragmenisasi perangkat, keamanan yang kadang-kadang menjadi perhatian.

b. iOS:

- Sistem operasi eksklusif untuk perangkat Apple (iPhone, iPad).
- Kinerja yang optimal, antarmuka yang intuitif, ekosistem aplikasi yang terkurasi.
- Kelebihan: Kinerja tinggi, keamanan yang baik, ekosistem aplikasi yang terkelola.
- Kekurangan: Pilihan perangkat terbatas, harga perangkat yang relatif mahal.

3. Sistem Operasi Server

a. Linux:

- Banyak distribusi Linux yang dirancang khusus untuk server (Ubuntu Server, CentOS, Red Hat Enterprise Linux).
- Stabilitas, keamanan, kemampuan untuk menjalankan berbagai layanan.
- Kelebihan: Gratis, sumber terbuka, dapat diandalkan, fleksibel.
- Kekurangan: Kurva pembelajaran yang lebih curam.

b. Windows Server:

- Versi Windows yang dirancang untuk server.
- Menawarkan fitur-fitur seperti Active Directory, Hyper-V, dan IIS.
- Kelebihan: Integrasi yang baik dengan lingkungan Windows, dukungan yang luas.
- Kekurangan: Biaya lisensi yang mahal.

4. Sistem Operasi Embedded

a. Sistem operasi real-time:

- Digunakan pada perangkat yang membutuhkan respon waktu yang sangat cepat (sistem kontrol industri, robotika).
- Contoh: VxWorks, QNX.

b. Sistem operasi untuk perangkat IoT:

- Digunakan pada perangkat Internet of Things (smart home devices, wearable devices).
- Contoh: FreeRTOS, Zephyr.

B. Instalasi Sistem Operasi

Instalasi sistem operasi adalah proses memasukkan dan mengkonfigurasi sistem operasi baru ke dalam sebuah komputer. Proses ini biasanya dilakukan saat membeli komputer baru, mengganti hard disk, atau ketika ingin menginstal ulang sistem operasi yang sudah ada.

Mengapa Perlu Instalasi Ulang?

- Performa menurun: Sistem operasi yang sudah lama digunakan seringkali menjadi lambat karena banyaknya file sampah dan program yang tidak terpakai.
- Masalah sistem: Terjadinya error, crash, atau infeksi virus yang sulit diatasi.

- Upgrade ke versi terbaru: Ingin menikmati fitur-fitur baru dari sistem operasi yang lebih modern.
- Mengganti sistem operasi: Ingin beralih dari satu sistem operasi ke sistem operasi lain (misalnya, dari Windows ke Linux).

Persiapan Sebelum Instalasi

- Media instalasi: USB flash drive atau DVD yang berisi file instalasi sistem operasi.
- Kunci produk: Jika sistem operasi berbayar, siapkan kunci produknya.
- Backup data: Sebelum menginstal, pastikan semua data penting telah dicadangkan.
- Spesifikasi komputer: Pastikan spesifikasi komputer Anda memenuhi syarat untuk menjalankan sistem operasi yang ingin diinstal.

Langkah-langkah Umum Instalasi

1. Buat media instalasi: Buat USB bootable atau DVD instalasi menggunakan alat yang disediakan oleh vendor sistem operasi.
2. Ubah pengaturan BIOS: Atur BIOS komputer agar boot dari media instalasi terlebih dahulu.
3. Mulai instalasi: Nyalakan komputer dan ikuti petunjuk yang muncul di layar.
4. Pilih bahasa dan pengaturan: Pilih bahasa, format waktu, dan tata letak keyboard yang sesuai.
5. Pilih jenis instalasi: Pilih apakah akan melakukan instalasi bersih (menghapus semua data) atau upgrade (memperbarui sistem operasi yang sudah ada).
6. Partisi hard disk: Bagi hard disk menjadi beberapa partisi sesuai kebutuhan.
7. Instalasi: Tunggu proses instalasi selesai.
8. Konfigurasi: Setelah instalasi selesai, lakukan konfigurasi awal seperti membuat akun pengguna, mengatur jaringan, dan menginstal driver perangkat keras.

C. Peran Sistem Operasi

Bayangkan komputer sebagai sebuah kota yang sibuk. Sistem operasi adalah polisi lalu lintas yang mengatur semua aktivitas di kota itu. Ia mengatur lalu lintas kendaraan (proses), memastikan setiap kendaraan (program) mendapatkan jalan (sumber daya komputer) yang cukup, dan memastikan semuanya berjalan lancar dan tertib.

Peran Utama Sistem Operasi

1. Manajemen Sumber Daya:
   - Prosesor: Membagi waktu penggunaan prosesor secara adil antara berbagai program yang sedang berjalan.
   - Memori: Mengalokasikan dan mendealokasikan memori untuk program-program yang sedang berjalan.
   - Perangkat Input/Output: Mengontrol perangkat seperti keyboard, mouse, monitor, printer, dan sebagainya.
   - File: Mengorganisir, menyimpan, dan mengambil data dari media penyimpanan seperti hard disk.
2. Antarmuka Pengguna:
   - Menyediakan tampilan: Memberikan tampilan visual yang memungkinkan pengguna berinteraksi dengan komputer.
   - Menerjemahkan perintah: Menerjemahkan perintah pengguna menjadi instruksi yang dapat dipahami oleh komputer.
3. Jaringan:
   - Konektivitas: Memungkinkan komputer terhubung dengan jaringan lain, seperti internet.
   - Protokol jaringan: Mengimplementasikan protokol jaringan seperti TCP/IP untuk komunikasi data.

4. Keamanan:

- Otentikasi: Memverifikasi identitas pengguna.
- Autorisasi: Mengontrol akses pengguna ke sumber daya sistem.
- Perlindungan data: Melindungi data pengguna dari akses yang tidak sah.

Contoh Peran Sistem Operasi dalam Kehidupan Sehari-hari

- Membuka dokumen: Saat Anda membuka dokumen Word, sistem operasi akan mencari file dokumen tersebut, mengalokasikan memori untuk aplikasi Word, dan memuat dokumen ke dalam memori.
- Menyimpan file: Saat Anda menyimpan file, sistem operasi akan mencari ruang kosong di hard disk dan menuliskan data file tersebut.
- Menghubungkan ke internet: Saat Anda menjelajahi internet, sistem operasi akan mengelola koneksi jaringan, mengirimkan permintaan data ke server, dan menerima respons dari server.
- Mencetak dokumen: Saat Anda mencetak dokumen, sistem operasi akan mengirimkan data ke printer dan mengontrol proses pencetakan.

Mengapa Sistem Operasi Penting?

- Kemudahan penggunaan: Sistem operasi membuat komputer lebih mudah digunakan dengan menyediakan antarmuka yang intuitif.
- Efisiensi: Sistem operasi mengoptimalkan penggunaan sumber daya komputer sehingga program dapat berjalan dengan lancar.
- Keamanan: Sistem operasi melindungi komputer dari ancaman seperti virus dan hacker.

Bahan Referensi

Informatika untuk SMA/MA kelas X. erlangga. Jakarta. 2. Mustofa, dkk, 2021. Informatika SMA Kelas X.Pusat Kurikulum. Balitbang Kemendikbud Ristek.Jakarta.